MY 2nd PROGRESS REPORT

Agent Capability Graph

Building an RDF Knowledge Graph to Discover MCP Servers and Agent Skills

Main Supervisor:	Raphael Troncy
Secondary Supervisor:	Thibault Ehrhart
Document Type:	Progress Report
Phase:	Dataset and RDF Pipeline Prototype

November 11, 2026

1 Project goal

The project explores how an RDF knowledge graph can help users discover and compare MCP servers, their tools, and Agent Skills. The graph is intended to connect source-backed descriptions with normalized capability information, while keeping track of where each metadata value came from. The current work is a prototype and a curated sample, rather than a general-purpose web crawler.

2 Dataset collection

The expanded sample contains ten MCP servers, forty-three selected documented tools, and ten Agent Skills. The records come from nine GitHub repositories. The MCP sample includes the official MCP server repository and projects from GitHub, Brave, Cloudflare, and AWS. The Skills sample includes repositories from Anthropic, Vercel, Microsoft, and Google.

Collected item	Count
MCP servers	10
Documented tool samples linked to servers	43
Agent Skills	10
Distinct source repositories	9
Downloaded Skill package archives	10

The collector records repository revisions, source URLs, field values, and evidence statuses in SQLite and exports a CSV for review. It saves the reviewed MCP README files and downloads each selected Skill directory as a ZIP package. The downloaded files are not executed. The tool list is a documented sample transcribed from the selected source material; it is not guaranteed to include every tool available from a server at runtime.

3 Ontology and graph model

The first ontology version is written in Turtle in `docs/acg_ontology.ttl`. It defines classes for MCP servers, MCP tools, Agent Skills, capabilities, source documents, metadata observations, and capability assertions. It also defines relationships such as `acg:exposesTool`, which connects a server to its tools, and provenance properties that record reviewed sources and observation status.

The ontology reuses terms from Schema.org, SKOS, PROV-O, RDF, RDFS, and OWL where appropriate. Its namespace, `urn:acg:ontology:`, is still provisional. The model distinguishes source descriptions from curator-assigned capability labels and preserves source evidence for later review.

4 SQLite-to-RDF pipeline

The current workflow has two main steps:

1. The expanded collector harvests the curated sample and writes the SQLite database and CSV under the interim-data directory.

2. The RDF generator maps the records to the ontology and writes the harvested Turtle graph under the processed-data directory.

The generated graph currently contains 2,210 RDF triples representing 63 harvested entities: 10 servers, 43 tools, and 10 Skills. Each collected field is retained as a source-backed metadata observation. The conversion also links each tool to its parent server and emits direct RDF properties for selected fields.

The generator parses both the ontology and the output graph, and checks that used ACG classes and properties are declared, that tools have server links, and that metadata observations are complete. These application-level checks are useful for the current pipeline, but they are not a full OWL reasoning step or SHACL validation. SHACL remains a possible later validation step, not a current project requirement.

5 Project organization

The code is organized in `src/acg/`. Small root-level launchers preserve the commands used during earlier development. Data is separated into `data/sources/` for maintained inputs, `data/raw/downloads/` for retrieved source packages, `data/interim/` for the SQLite and CSV collection outputs, and `data/processed/` for the generated RDF graph. Project reports and ontology documents are kept under `docs/`. A `.gitignore` file excludes local Python environments, cache files, and downloaded packages.

6 Current limitations and next steps

The current sample and pipeline provide a working base for evaluating the ontology, but several steps remain:

- Refine competency questions from the perspective of graph users and state the expected answer for each question.
- Test whether the generated graph can answer those questions using SPARQL queries.
- Improve direct RDF mappings for useful metadata such as repository identity, Skill compatibility, and package archive location. These values are currently preserved as observations when no dedicated mapping is applied.
- Review the sample and its source evidence, and distinguish clearly between source-declared facts and curator interpretations.
- Consider SHACL shapes after the ontology and data fields stabilize.

7 Conclusion

The project has progressed from a small exploratory JSON-LD example to a multi-source dataset and a reproducible SQLite-to-RDF prototype. The next milestone is to connect the ontology and graph to concrete user questions, then refine the model and mappings based on the answers those questions require.